Microservice Project
Installation & Setup Guide

Project Members:

- **Batuhan Orkun İnce** (212010020015)
- **Seyfullah Adıgüzel** (212010020102)

1. Prerequisites

Before starting, ensure you have the following installed on your machine:

- **Docker & Docker Compose:** Required to run the services in isolated containers. (Download Docker)
- **Git:** Required to clone the repository. (Download Git)

2. Project Architecture

The project consists of three main components:

1. **Eureka Server (Port 8761):** Acts as the Service Registry. All other microservices register themselves here so they can discover each other.
2. **Product Service (Port 8081):** A microservice that manages products. It provides a simple UI to list available products.
3. **Order Service (Port 8082):** A microservice that handles orders. It communicates with the Product Service via Eureka to fetch product data. It is equipped with a **Circuit Breaker (Resilience4j)** to prevent cascading failures.

3. Installation & Running the Project

Step 1: Clone the Repository

Open your terminal (or Command Prompt/PowerShell) and run the following command to clone the project:

```
git clone https://github.com/SEYFULLAH01/microservice-project.git
cd microservice-project
```

Step 2: Build and Run with Docker Compose

Since the project is fully dockerized, you do not need to manually install Java or Maven. Docker will handle the compilation and deployment.

Run the following command in the root directory (where `docker-compose.yml` is located):

```
docker compose up -d --build
```

Note: The `-d` flag runs the containers in the background (detached mode), and `--build` ensures that the latest code changes are compiled.

Step 3: Verify the Deployment

Wait about 30-40 seconds for all services to start up and register themselves. You can verify that the containers are running by typing:

```
docker ps
```

4. Accessing the Services

Once the deployment is complete, you can access the following interfaces via your web browser:

- **Eureka Service Registry:** http://localhost:8761
  Here you can see the registered instances (`PRODUCT-SERVICE` and `ORDER-SERVICE`).
- **Product Service UI:** http://localhost:8081
  Displays the list of available products.
- **Order Service UI:** http://localhost:8082
  Provides an interface to place an order, which fetches data from the Product Service.

5. Testing the Circuit Breaker (Resilience4j)

The project includes a robust fault-tolerance mechanism. You can test it by simulating a service failure:

1. Stop the Product Service manually:

```
docker compose stop product-service
```

2. Go to the **Order Service UI** (http://localhost:8082) and click the "Get Orders" button.
3. You will instantly receive a **Fallback Message** (*"Üzgünüz, Ürün Servisinde bir arıza var..."*). The system does not crash; it elegantly handles the failure.
4. Restart the Product Service:

```
docker compose start product-service
```

5. Wait about 10 seconds and try again. The system will automatically heal (Self-Healing) and return the successful response.

6. Testing Client-Side Caching (Single Point of Failure Prevention)

This project utilizes **Client-Side Caching** so that the Eureka Server is not a single point of failure. You can demonstrate that the microservices can still communicate even if the Eureka Registry crashes.

1. Ensure all services are running and the Order Service works properly.
2. Stop the Eureka Server manually:

```
docker compose stop eureka-server
```

3. Go back to the **Order Service UI** (http://localhost:8082) and click the "Get Orders" button.
4. **The request will still succeed!** The Order Service uses its local cache of the registry to find the Product Service, proving the system is highly available.
5. To demonstrate when it finally fails, forcefully remove the Product Service and restart it without waking up Eureka (so it gets a new IP that the Order Service doesn't know about):

```
docker compose rm -fsv product-service
docker compose up -d --no-deps product-service
```

6. Try placing an order again. This time it will fail, demonstrating the true purpose of the Eureka Registry. Finally, start Eureka again to restore the system:

```
docker compose start eureka-server
```